AI with Cursor — alcance del curso

Seis clases de 90 minutos, una detrás de otra, para 4 personas. Cada quien trabaja en el proyecto que ya mantiene.

Los nombres de abajo son los del glosario del curso. Cada clase enseña esos conceptos y, cuando corresponde, los deja escritos en el repositorio. Se habla en español. Los archivos que se guardan van en inglés. No hay tareas entre clases.

Cursor es el laboratorio. Los conceptos valen igual en Copilot o Claude: cambia la pantalla, no la idea.

Dónde arrancamos

El primer día el proyecto ya está abierto. No se crea uno nuevo.

La sesión 1 cubre los fundamentos y el primer uso, todavía sin mapa ni reglas nuevas:

- El modelo (**LLM**, C01) predice texto. No conoce el repositorio.
- Solo ve el **contexto** de esta petición (C04). El cupo es finito: la **ventana de contexto** (C02).
- Una respuesta segura puede ser falsa: **alucinación** (C03).
- Cada chat empieza en blanco: **sin memoria entre sesiones** (C05). Lo que debe durar va en git.
- El **prompt** (C06) trae objetivo, contexto, límites y cómo verificar.
- El **agente** (C08) puede editar archivos y usar la terminal. La persona aprueba (**human-in-the-loop**, C11) y corre la prueba (**verificación y evidencia**, C12).
- El **alcance explícito** (C09) dice qué sí entra y qué no. **Explorar** no es **editar** (C10).

En esa misma clase piden un cambio chico, con pocos archivos, y lo prueban ellos. También eligen con su lead el ticket de uno o dos días que van a recorrer después.

Dos ideas se nombran hoy y se escriben después: las **instrucciones persistentes** (C07) y el **contexto en capas** (C13).

Dónde terminamos

Al cerrar la sesión 6, cada persona recorrió un ticket con el **ciclo con gates** (C18):

intake → analysis → implement → validate → review → close

Eso es: contrato escrito → análisis → cambio mínimo → prueba que otra persona puede repetir → revisión contra el contrato → cierre en el repositorio.

El proyecto queda con **contexto en capas** (C13):

1. **Código** — el que ya tenían.
2. **Instrucciones persistentes** (C07) — reglas que el editor carga antes del prompt.
3. **Memoria en archivos** — mapa (C14), historia del ticket (C15) y contexto de producto (C24).

La persona puede repetir ese ciclo en el próximo ticket.

Las seis clases

Los conceptos de cada fila son los de la tabla del glosario. La frase es la definición del curso.

Sesión 1 — Fundamentos y primera práctica

Concepto	Definición
C01 LLM	Predice texto token a token; no conoce tu repo.
C02 Ventana de contexto	Cupo finito de tokens por petición.
C03 Alucinación	Respuesta plausible pero incorrecta o inventada.
C04 Contexto	Todo lo que el modelo ve en esta petición.
C05 Sin memoria entre sesiones	Cada chat empieza en blanco; git es la memoria real.
C06 Prompt	Instrucción: objetivo, contexto, límites, formato, verify.
C08 Agente	LLM con permiso para actuar: editar, terminal, buscar.
C09 Alcance explícito	Qué sí y qué no entra al contexto ni al diff.
C10 Explorar / editar / planificar	Separar preguntar, escribir código y diseñar.
C11 Human-in-the-loop	La persona aprueba, verifica y corrige.
C12 Verificación y evidencia	Prueba real documentada; no el "listo" del chat.

C07 y C13 se nombran aquí y se materializan desde la sesión 2.

Sales con: un cambio chico, alcance explícito y una verificación corrida por ti. Todavía sin carpeta `cursor/`.

Sesión 2 — Mapa e instrucciones persistentes

Concepto	Definición
C07 Instrucciones persistentes	Reglas que la herramienta carga antes de tu prompt.
C13 Contexto en capas	Código, reglas y memoria en archivos, por separado.
C14 Mapa del repositorio	Índice en git: dónde vive cada cosa.

Sales con: `context-map.md`, `project.config.json`, un `project-context.md` del stack real (cómo se agrega algo y cómo se prueba) y reglas en `.cursor/rules/`. La prueba: preguntar dónde va un test y que la respuesta cite el mapa, no una carpeta inventada.

Sesión 3 — Memoria durable e intake

Concepto	Definición
C05 Sin memoria entre sesiones	El acuerdo del chat no sobrevive. Hay que escribirlo.
C15 Memoria durable	El estado de la historia vive en archivos, no en el hilo.
C16 Contrato de scope	Story y manifest antes de codear.
C17 Análisis previo	Plan escrito: archivos y riesgos.
C18 Ciclo con gates	Hoy solo se cruzan intake y analysis.

Sales con: `cursor/analysis/features/<slug>/` con `user-story.md`, `feature-manifest.md` (in scope, out of scope, definition of done) y `analysis.md`. Sin código de producto.

Sesión 4 — Implementar y validar

Concepto	Definición
C03 Alucinación	"El test pasó" en el chat no es evidencia.
C11 Human-in-the-loop	Lees el diff. Lo que está fuera del contrato se revierte.
C12 Verificación y evidencia	Corres el comando, o dejas el paso manual escrito.
C18 Ciclo con gates	Hoy: implement y validate. Review es la sesión 5.
C19 Implementación acotada	El diff solo cubre el contrato.
C20 Validación reproducible	Un checklist que otra persona ejecuta sin tu chat.

Sales con: el cambio mínimo, `implementation-notes.md` y `test-checklist.md`. No se mergea.

Sesión 5 — Review, cierre y ventana

Concepto	Definición
C02 Ventana de contexto	Un chat largo pierde foco y mezcla decisiones viejas.
C21 Gestión de ventana	Un chat nuevo por fase. La memoria sigue en los archivos.
C22 Review estructurado	Revisar el diff contra story y manifest.
C23 Cierre y handoff	El estado final queda en el repo, no en el chat.

Sales con: findings de review (o confirmación de que el diff cumple el contrato), manifest e índice actualizados, y la regla `context-scope.mdc`.

Sesión 6 — Contexto de producto

Concepto	Definición
C24 Contexto de producto	Visión y backlog, separados del ticket del día.

La automatización del ciclo (C27) se menciona como opt-in. No es entregable de la clase.

Sales con: una página de visión, tres ítems de backlog `FW-*` (distintos del ticket de Jira o GitHub) y el checklist para repetir el ciclo.

Qué queda escrito para el agente

Arquitectura del repo, estándares y convenciones sí entran. Se escriben como contexto en capas (C13), no como un manual largo.

Capa	Concepto	Qué queda en el repo	Qué le dice al agente
Código	—	El proyecto que ya tienen	La fuente de verdad
Reglas	C07 Instrucciones persistentes	`.cursor/rules/`	Estándares que se cargan solos: alcance, secretos, idioma de la documentación
Memoria	C14 Mapa del repositorio	`context-map.md` y `project-context.md`	Dónde vive cada parte y cómo se trabaja en ese stack: puntos de entrada, convenciones, cómo probar
Memoria	C15 + C16 + C17	Carpeta de la historia: story, manifest, analysis	En este ticket, qué entra, qué no, qué archivos y qué riesgos

Capa	Concepto	Qué queda en el repo	Qué le dice al agente
Memoria	C20	`test-checklist.md`	Cómo repetir la prueba
Memoria	C24 Contexto de producto	Visión corta y backlog `FW-*`	Para quién es el producto y qué queda para después

Las convenciones y los patrones de código que el equipo quiere usar van en las instrucciones persistentes (C07) y en el `project-context.md` (C14), solo los que importan al programar. Si ya hay documentación larga, estos archivos apuntan a ella. Meterla entera satura la ventana de contexto (C02).

En el glosario, "patrón" también nombra los hábitos de uso (C09–C24): alcance explícito, human-in-the-loop, ciclo con gates. Esos se practican en clase. No son un catálogo de patrones de diseño.

Antes del primer día

Cuatro cosas. Si falta alguna, se resuelve antes de arrancar. No son tareas del curso.

1. **Cursor instalado y el proyecto abierto.** El proyecto en el que ya trabajan. Copilot o Claude sirven si ya los usan: el concepto es el mismo.
2. **Permiso para guardar en una rama.** Branch y commit. El merge sigue el flujo del equipo, después del review (C22), no en la sesión 4.
3. **Un ticket pequeño, acordado con el lead.** Un bug o un cambio de uno o dos días. Es el ejercicio de las sesiones 3 a 5 (C16–C23). Si es un épico, el lead recorta el alcance (C09) antes de empezar.
4. **Cómo se verifica hoy.** El comando que ya corren (`npm test`, `go test`, lint, build) o los pasos a mano. Esa es la verificación (C12) desde la sesión 1.

Cómo es

Personas	4
Sesiones	6, seguidas
Duración	90 minutos cada una
Entre clases	Nada para la casa. Lo que no cabe se recorta en esa sesión
Práctica	En el repositorio de cada quien
Idioma	Español en clase. Inglés en los archivos que se guardan

Cada clase: el concepto, práctica en el teclado, cierre con el artefacto de esa sesión.

Qué queda fuera

El glosario lo marca como no cubierto, o solo como mención:

- Proyecto nuevo desde cero.
- Copiar carpetas de otro producto.
- RAG, fine-tuning, multi-agente y guardrails avanzados.
- Dejar escrita toda la arquitectura de la compañía. Se escribe el mapa (C14), las reglas (C07) y el contrato del ticket (C16).